ASSIGNMENT 1

NAME:P.B.VAMSI KRISHNA
192525147

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: 11-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is

at K5 level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/VAMSI00927/DSA0120--C-.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time
complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least
Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

• The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each

• How the same problem would be written in a procedural style, and what changes in an object-oriented style

• The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system

• The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)
You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3

Marks Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one

place only. Discuss:

• The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples

• #define, const and constexpr constants, and why constexpr is preferred for the fare

table • enum and enum class for zones, and why enum class is safer

• Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

• The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
• Why money should be stored in integer paise and not as double rupees

• Type compatibility rules, and what the compiler does with int, double and bool values in the same expression

• The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

• Integer division and integer-to-double conversion

• Comparison of signed and unsigned values, and the compiler warning it

produces • Narrowing conversions, and how brace initialisation (int x{3.7};)

prevents them • static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


int main()

{

 int a = 7, b = 2;
```

```
 double c = a / b;
 double d = static_cast<double>(a) / b;

 std::cout << c << ' ' << d << '\n';


 char ch = 'A' + 2;

 std::cout << ch << ' ' << ch + 1 << '\n';


 bool flag = 5;

 std::cout << flag << ' ' << std::boolalpha << flag << '\n';


 unsigned int u = 1;

 int s = -1;

 std::cout << (s < u ? "s < u" : "s >= u") << '\n';

}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate
Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


int main()

{

 int x = 5, y = 3;

 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';


 int z = x++ + ++y;

 std::cout << z << ' ' << x << ' ' << y << '\n';


 std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';


 int k = 0;

 for (int i = 0; i < 10; i += 3) k += i;

 std::cout << k << '\n';

}
```

My predicted output (write before running):
For every line, name the operators involved and the order in which precedence and

associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's

journeys. Discuss:

• When to use if–else, a switch on the zone, and the conditional operator

• How a for loop, a range-based for loop and a while loop could process the journeys, and which is best

• Where break and continue are needed (for example, skipping an invalid

journey) • What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>


enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs

20, Rs 30 constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card


long long fare(Zone z, bool peak)

{
```

```
 // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE,
then add  // PEAK_SURCHARGE_PERCENT % in peak hours using
integer arithmetic only
}

bool canTravel(long long balance, long long f)
{
 // TODO 2: Return true only if the balance left after paying the fare is at
least  // MIN_BALANCE_PAISE

}

int main()
{
 long long balance = 10000; // Rs 100.00 in paise
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle,
false}};  for (const Trip& t : trips) {
 long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
 balance -= f;
 std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)";
 }
 std::cout << " balance Rs " << balance / 100 << '\n';
 }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

• A procedural version with separate variables and free functions

• An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()

• How the class prevents a negative balance

• Which version is easier to change when the minimum balance

changes Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom’s Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

• Reading values with cin and checking cin.fail() when the operator types letters instead of numbers

• Clearing the error state with cin.clear() and discarding the bad input

• Formatting money with <iomanip> (setw, setfill, fixed, setprecision)

• Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom’s Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

• Normal journeys in each zone, peak and off-peak

• Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly

Rs 50 • Invalid cases: an unknown zone, a negative recharge, letters typed for the hour

• A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

• Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ

• Whether int overflows for the network total

• The compiler warnings you received with -Wall -Wextra and what each

meant • Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

• The types, constants and classes you recommend and why

• How a fare revision will be carried out

• The limitations of your design

• Whether the design remains correct for the whole network
Your recommendation must be based on evidence from your own

experiments. Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your

original solution. A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are

affected. Evidence Expected

A clear analysis of how the changed requirements affect the original

design. B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.
MARK DISTRIBUTION

Task Bloom's Level Marks

Task 1 – Object-Oriented Analysis K5 – Evaluate 4

Task 2 – Tokens and Constants K5 – Evaluate 3

Task 3 – Data Types and Compatibility K5 – Evaluate 4

Task 4 – Implicit Conversions K5 – Evaluate 4

Task 5 – Operators and Precedence K5 – Evaluate 4

Task 6 – Control Structures K5 – Evaluate 4

Task 7 – Procedural vs Object-Oriented K5 – Evaluate 4

Task 8 – Input and Output K5 – Evaluate 4

Task 9 – Implementation & Testing K5 – Evaluate 4

Task 10 – Expectation vs Execution K5 – Evaluate 4

Task 11 – Final Recommendation K5 – Evaluate 4

Subtotal 42

Task 12 – Day-4 Design Challenge K5 – Evaluate 8

TOTAL 50

Submission Requirements

• GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ANSWERS:

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

Object-oriented analysis identifies the important entities in the real metro domain and assigns each entity its own data and responsibilities. The design is not merely a collection of variables; it groups state with the operations that maintain valid state.

Object / class	Important data	Behaviour / responsibility	Justification
Passenger	Passenger ID, category (regular/student/senior)	Provide identity and concession category	Concession eligibility belongs to the passenger, not to the card balance.
SmartCard	Card ID, private balance in paise, linked passenger	canTravel(fare), deduct(fare), recharge(amount)	The card must protect the Rs 50 reserve and prevent negative balance.
Journey	Origin/destination or farthest zone, hour, day type	Validate journey details; expose zone and time	Each trip has independent facts used to calculate its fare.
Zone	Zone identifier/name	Represent Inner, Middle, Outer (later Airport)	A typed zone avoids confusing arbitrary integers.
FareTable	Base fare by zone, surcharge/discount rules	Return the fare in paise for a valid journey	Centralises fare policy so a revision is made in one place.

Procedural style versus object-oriented style

A procedural program would store balances and fares in arrays or global variables and call free functions such as calculateFare(zone, hour), canTravel(balance, fare) and deduct(balance, fare). This can be correct for a small program, but every function must receive the correct balance and all code must follow the same validation rules. A caller could accidentally modify a global balance directly. In an object-oriented program, SmartCard owns its balance and exposes controlled operations. FareTable owns the fare policy, and Journey stores the trip facts. The public interface hides implementation details. A change to the minimum reserve is made in one named constant or policy object rather than in several unrelated functions.

Four basic OOP principles in this system

Principle	Application	Benefit
Encapsulation	SmartCard::balancePaise is private; deduction is allowed only through deduct().	No unrelated code can directly set a negative or invalid balance.
Abstraction	The gate calls card.canTravel(fare) and card.deduct(fare) without knowing internal checks.	The gate uses a simple interface instead of managing balance arithmetic.
Inheritance	A Passenger base class may have StudentPassenger and SeniorPassenger derived classes if their policies genuinely differ.	Shared identity is reused while category-specific policy can be added.
Polymorphism	A virtual concessionRate() or calculateConcession() can be overridden by StudentPassenger and SeniorPassenger.	The gate can apply a concession through a common interface.

Journey invariants / rules

- A zone must be one of the recognised zone values.

- The hour must be between 0 and 23 and minutes between 0 and 59.
- The peak surcharge applies only during 08:00–09:59 and 17:00–19:59.
- Fare must be non-negative and represented in paise.
- A successful journey must leave at least 5,000 paise (Rs 50).
- A denied or invalid journey must not change the card balance.
- A deduction must never exceed the available balance and must not make the balance negative.
- All fare rules must be defined centrally; duplicated magic numbers are prohibited.

Alternative concept and complexity analysis

In a procedural solution, global arrays hold balances and fares. A function calculates the fare and another checks and updates a selected balance. For one journey, fare calculation and balance checking take constant time, O(1). The memory required for one card balance is also O(1). The same is true in the object-oriented version: a SmartCard object stores a balance and performs constant-time operations. OOP does not make the arithmetic faster; it improves ownership, safety and maintainability.

If the minimum balance changes, a procedural design with the reserve repeated in multiple functions may require edits in several places. A well-designed OOP solution changes one constant or policy. If both designs use one central constant, the number of edits may be similar; the advantage comes from enforcing a single controlled interface.

Conclusion: Use SmartCard, Journey and FareTable because they represent real domain concepts and make the balance and fare rules easier to protect and extend.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

C++ source code is made from tokens. The following examples are taken from the metro fare program.

Token category	Meaning	Examples
Keywords	Reserved words with language-defined meaning.	class, private, public, enum, constexpr, return, if, switch, long, bool
Identifiers	Programmer-chosen names.	SmartCard, balancePaise, calculateFare, PEAK_START_1
Constants / literals	Fixed values written directly or declared as constants.	5000, 1000LL, true, 'A', "Denied"
Operators	Symbols that perform operations.	+, -, *, /, %, ==, >=, &&, ||, +=, ?:, <<
Punctuators	Symbols that structure declarations and expressions.	;, { }, ( ), [ ], :, ,, .

Recommended named constants

```
constexpr long long RUPEES_TO_PAISE = 100;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

```
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int PEAK_START_MORNING = 8;
constexpr int PEAK_END_MORNING = 10;   // exclusive
constexpr int PEAK_START_EVENING = 17;
constexpr int PEAK_END_EVENING = 20;   // exclusive
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
```

constexpr is preferred for compile-time fare values because it is type-checked, scoped, usable in constant expressions and easy to inspect. const means an object cannot be modified after initialisation, but it is not necessarily required to be evaluated at compile time. #define performs textual substitution before compilation and has no C++ type or normal scope; it can create precedence surprises, for example #define RATE 1+2 and 3*RATE expands to 3*1+2.

enum versus enum class

```
enum ZoneOld { Inner, Middle, Outer };            // names enter surrounding scope
enum class Zone { Inner, Middle, Outer };         // scoped, strongly typed
Zone z = Zone::Outer;
```

enum class is safer because values are scoped (Zone::Outer), do not implicitly convert to int, and cannot be accidentally mixed with unrelated enumerations. A fourth zone can be added explicitly as Zone::Airport.

Identifier naming rules

- An identifier may contain letters, digits and underscores, but cannot start with a digit.
- Keywords cannot be used as identifiers.
- Names are case-sensitive: balance and Balance are different.
- Use PascalCase for class/type names (SmartCard), camelCase for functions and variables (calculateFare, balancePaise), and uppercase snake case for constants (MIN_BALANCE_PAISE).
- Choose descriptive names and avoid unexplained abbreviations or magic numbers.

Alternative concept and complexity analysis

Replacing constexpr values and enum class with #define macros removes type checking and scope. A macro usually occupies no storage, while a constexpr value is commonly folded by the compiler and also often occupies no runtime storage unless an address or object is required. Neither approach materially changes the O(1) fare lookup time. The important difference is correctness: constexpr and enum class provide compiler checking, clearer diagnostics and safer maintenance.

TASK 3 – Choose Data Types and Check Type Compatibility (4 Marks)

The table below gives suitable types. Exact sizeof results are implementation-dependent; the listed sizes are typical for a 64-bit g++ environment and must be confirmed with sizeof on the student's compiler.

Value	Recommended type	Typical size	Reason / range
Fare in paise	long long	8 bytes	Exact integer arithmetic; signed 64-bit range is approximately −9.22×10^18 to +9.22×10^18.
Card balance in paise	long long	8 bytes	Supports large stored values and safe arithmetic.
Hour / minute	int	4 bytes	Small bounded whole-number values; validate 0–23 and 0–59.
Zone	enum class Zone	typically 4 bytes	Strongly typed set of named zones.
Journey count	long long	8 bytes	Can represent millions and larger counts.
Peak flag	bool	typically 1 byte	Only true or false.
Passenger category initial	char	1 byte	Useful only for a compact character code; enum class is better.
Percentage	int	4 bytes	Whole-number percentage such as 20.
Measurement / optional ratio	double	8 bytes	Use for non-money measurements where fractional values are needed.
Loop index	int or size_t	typically 4 or 8 bytes	Use int for bounded small loops; size_t for container sizes.

Why integer paise is safer than double rupees

Binary floating-point cannot represent many decimal fractions exactly. For example, 0.1 has a repeating binary representation. Repeated additions can produce a value such as 0.30000000000000004 rather than exactly 0.30. Integer paise stores Rs 0.10 as exactly 10 and Rs 0.30 as exactly 30. The fare logic therefore uses integer paise and integer percentage arithmetic.

```
long long farePaise = 3000;
long long peakFarePaise = farePaise * (100 + 20) / 100; // 3600 paise
```

Type compatibility and conversions

When int and double are used in one arithmetic expression, the int is normally converted to double and the operation is performed as double. When bool is used in an arithmetic expression, false behaves as 0 and true as 1 after integral promotion. Assigning a non-zero integer to bool produces true; assigning zero produces false. Integer division between two integers discards the fractional part toward zero.

CLIMITS and network totals

```
#include <climits>
std::cout << INT_MAX << '\\n';
std::cout << LLONG_MAX << '\\n';
```

A typical 32-bit int has INT_MAX = 2,147,483,647. If 3,000,000 journeys each cost Rs 30, the maximum simple total is Rs 90,000,000 = 9,000,000,000 paise. This exceeds INT_MAX when stored as paise, so int is not sufficient for the network's paise total. A signed 64-bit long long is easily sufficient for this scale.

Journeys	At Rs 30 per journey	Total in paise	Fits 32-bit int paise?

20,000	Rs 600,000	60,000,000	Yes
500,000	Rs 15,000,000	1,500,000,000	Yes
3,000,000	Rs 90,000,000	9,000,000,000	No

Alternative concept and complexity analysis

Using double rupees would still use O(1) time per addition and typically 8 bytes per value, just like a 64-bit long long. The difference is numeric exactness: double can represent a wide range but not every decimal amount exactly. The error depends on the sequence and magnitude of values. Integer paise has no rounding error for whole-paise fares and avoids overflow at the given network scale when long long is used.

Evidence: reproducible precision check

```
#include <iomanip>
#include <iostream>
int main() {
    double total = 0.0;
    for (int i = 0; i < 10; ++i) total += 0.1;
    std::cout << std::setprecision(17) << total << '\\n';
    long long paise = 0;
    for (int i = 0; i < 10; ++i) paise += 10;
    std::cout << paise << " paise\\n";
}
```

The floating-point output may show a value slightly different from 1.0, while the integer result is exactly 100 paise. Run this program and record the exact compiler output in the evidence section.

TASK 4 – Control Implicit Conversions (4 Marks)

Implicit conversions occur when the compiler converts a value without an explicit cast. They must be understood because they can change the result of fare calculations or comparisons.

Location / example	Conversion	Effect / safe approach
double x = 7 / 2;	Integer division occurs first; result 3 is then converted to double.	Use 7.0/2 or static_cast<double>(7)/2.
char ch = 'A' + 2;	char is promoted to int for addition; result is converted back to char.	Use a valid character range and explicit cast if needed.
bool flag = 5;	int converts to bool; non-zero becomes true.	Use true/false explicitly.
int s=-1; unsigned u=1; s<u	The signed int is converted to unsigned for the comparison when usual arithmetic conversions apply.	Avoid mixing signed and unsigned; validate before conversion.
int x{3.7};	Narrowing conversion is rejected by list initialisation.	Use an appropriate type or an explicit, justified cast.

Predicted output – Task 4 program

```
3 3.5
C 68
1 true
```

```
s >= u
```

Explanation: a/b is integer division (7/2 = 3), then 3 becomes 3.0. The explicit cast changes the first operand to double, so 7.0/2 = 3.5. 'A' has the character code 65 in the usual execution character set; adding 2 produces 'C'. In ch + 1, ch is promoted to int, so the numeric result is 68. Integer 5 converts to bool true; default stream formatting prints 1, and std::boolalpha prints true. In s < u, −1 is converted to unsigned int, becoming a very large unsigned value, so the condition is false and the conditional expression prints s >= u.

Corrected conversion-conscious code

```
int whole{7};
int divisor{2};
double exact = static_cast<double>(whole) / divisor;
bool valid{true};
char letter = static_cast<char>('A' + 2);
// Do not compare a possibly negative signed value with unsigned.
// Check the sign first, then convert only after validation.
int signedValue{-1};
unsigned int unsignedValue{1};
bool comparison = signedValue >= 0 &&
                  static_cast<unsigned int>(signedValue) < unsignedValue;
```

Alternative concept and complexity analysis

If every conversion were explicit, static_cast would document the programmer's intention and brace initialisation would reject narrowing at compile time. Running time and memory are normally unchanged because casts generally compile to the same machine operations or no operation. Compile-time diagnostics catch unsafe narrowing earlier, but explicit casts do not make an incorrect conversion logically safe; the programmer must still validate values.

Peak boundary check

Use hour >= 8 && hour < 10 for the morning period and hour >= 17 && hour < 20 for the evening period. This makes 10:00 and 20:00 off-peak. If minutes are included, compare total minutes since midnight so that 09:59 is peak and 10:00 is not.

TASK 5 – Write Correct Expressions with the Right Precedence (4 Marks)

Fare expressions

```
bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}
long long applyPeakSurcharge(long long basePaise) {
    return basePaise * (100 + PEAK_SURCHARGE_PERCENT) / 100;
}
// If a concession rate is supplied, apply it using integer arithmetic:
```

```
// farePaise = farePaise * (100 - concessionPercent) / 100;
```

Relational operators (>= and <) are evaluated before logical AND (&&), and logical AND is evaluated before logical OR (||). Parentheses are still recommended because they communicate the two separate time windows. Compound assignment such as balance -= fare is equivalent to balance = balance - fare, with the left-hand expression evaluated only once.

Increment and division rules

- x++ is post-increment: the expression yields the old value, then x increases.
- ++x is pre-increment: x increases first, and the expression yields the new value.
- Avoid i = i++ because it is confusing and can lead to undefined or version-dependent sequencing issues; use i++ as a separate statement.
- For integer division, the quotient is truncated toward zero. The remainder has the sign of the dividend: −10 / 3 is −3 and −10 % 3 is −1.

Predicted output – Task 5 program

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: multiplication has higher precedence than addition, so 5 + (3×2) = 11; parentheses force (5+3)×2 = 16. Line 2: x++ contributes old x=5, while ++y first changes y to 4; z=5+4=9, then x=6 and y=4. Line 3: conditional operator selects max(6,4)=6; remainder is evaluated before multiplication, so (6%4)×2=4. Line 4: integer quotient/remainder results are 3, 1, −3 and −1. Line 5: loop values 0, 3, 6 and 9 are added, producing 18.

Alternative concept and complexity analysis

A 24-element bool peak[24] table can be initialised once and queried by peak[hour]. Both the logical expression and array lookup are O(1). The table needs 24 bool elements (typically 24 bytes, although the language does not guarantee sizeof(bool)==1 on every implementation). The table may reduce repeated comparisons, but it adds initialisation and storage and must be updated carefully if time rules change. The expression is clearer for only two intervals.

TASK 6 – Design the Control Structures (4 Marks)

Use switch for selecting a base fare from a small fixed set of zones; use if–else for validating input and deciding whether a card can travel; use the conditional operator only for a short value selection. A for loop is suitable when the number of journeys is known. A range-based for loop is clearest when iterating over an array or container. A while loop is suitable when journeys continue until an operator enters a stop command.

SIMATS-style algorithmic pseudocode

```
ALGORITHM: COMPUTE_AND_PROCESS_FARES
INPUT: journeys, cardBalance, journeyCount
CONSTANT: MIN_BALANCE = 5000 paise
FOR each journey in journeys DO
    IF zone is not a recognised zone OR hour is outside 0..23 THEN
        DISPLAY "Invalid journey"
        CONTINUE
    ENDIF

    CASE zone OF
        INNER: baseFare ← 1000
        MIDDLE: baseFare ← 2000
        OUTER: baseFare ← 3000
        OTHERWISE: DISPLAY "Invalid zone"; CONTINUE
    ENDCASE

    IF (8 ≤ hour < 10) OR (17 ≤ hour < 20) THEN
        fare ← baseFare × 120 / 100
    ELSE
        fare ← baseFare
    ENDIF

    IF cardBalance − fare ≥ MIN_BALANCE THEN
        cardBalance ← cardBalance − fare
        DISPLAY fare and cardBalance
    ELSE
        DISPLAY "Denied (low balance)"
    ENDIF
END FOR
OUTPUT: final cardBalance
END ALGORITHM
```

Completed code for TODO 1 and TODO 2

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1; // invalid zone
    }
    if (peak) {
        base = base * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    }
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
```

```
            balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0') << f % 100
                      << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Expected output (matches the assignment):

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

At exactly 10:00 and 20:00 the peak flag must be false. At exactly 08:00 and 17:00 it must be true. The supplied code receives a bool peak value; the calling program must calculate that bool using the stated boundary expression.

Alternative concept and complexity analysis

An array indexed by zone gives direct O(1) access, just like a switch. The array occupies three long long values in the original version (typically 24 bytes). A switch may compile to a jump table or comparisons. A fourth zone requires adding a new enum value and a matching switch case, or extending the array and keeping enum-to-index mapping consistent. For a growing or configurable fare table, a validated array/map is easier to maintain; for three fixed zones, switch is explicit and readable.

TASK 7 – From Procedural to Object-Oriented (4 Marks)

Procedural version

```
#include <iostream>
constexpr long long MIN_BALANCE = 5000;
long long calculateFare(int zone, bool peak) {
    long long base = (zone == 0 ? 1000 : zone == 1 ? 2000 :
                      zone == 2 ? 3000 : -1);
    if (base < 0) return -1;
```

```
    return peak ? base * 120 / 100 : base;
}
bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= MIN_BALANCE;
}
bool deduct(long long& balance, long long fare) {
    if (!canTravel(balance, fare)) return false;
    balance -= fare;
    return true;
}
int main() {
    long long balance = 10000;
    long long f = calculateFare(2, true);
    if (deduct(balance, f))
        std::cout << "Approved; balance=" << balance << '\\n';
    else
        std::cout << "Denied; balance=" << balance << '\\n';
}
```

Object-oriented SmartCard version

```
#include <iostream>
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_BALANCE_PAISE = 5000;
public:
    explicit SmartCard(long long openingBalance)
        : balancePaise(openingBalance >= 0 ? openingBalance : 0) {}

    long long balance() const { return balancePaise; }

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }

    bool recharge(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        return true;
    }
};
int main() {
    SmartCard card(10000);
    if (card.deduct(3600))
        std::cout << "Approved; balance=" << card.balance() << '\\n';
    else
        std::cout << "Denied; balance=" << card.balance() << '\\n';
}
```

The private data member cannot be modified directly by other functions. The only deduction path checks the fare and the Rs 50 reserve before subtraction. The class therefore protects the invariant. The procedural version can enforce the same rule if all callers use deduct(), but a public global balance is easier to misuse.

Comparison and complexity

Aspect	Procedural	SmartCard OOP
State	Separate variables / references	Private balance inside object
Operations	Free functions receive balance	Member functions operate on owned balance
Time per check/deduction	O(1)	O(1)
Space per card	O(1)	O(1)
Safety	Depends on every caller using functions correctly	Encapsulation prevents direct external modification
Changing minimum reserve	Change central constant if properly centralised	Change one class policy/constant

If balance were public, time and space would remain O(1), but any part of the program could assign a negative balance or deduct below Rs 50. The issue is not performance; it is loss of invariant protection.

TASK 8 – Read and Print Data Safely (4 Marks)

The gate must not trust operator input. After cin >> value, test the stream state. If the operator enters letters where a number is required, cin.fail() becomes true. Call cin.clear() to reset the error flag and cin.ignore(numeric_limits<streamsize>::max(), '\\n') to discard the invalid line.

```
#include <iostream>
#include <limits>
#include <iomanip>

int readHour() {
    int hour{};
    while (true) {
        std::cout << "Enter hour (0-23): ";
        if (std::cin >> hour && hour >= 0 && hour <= 23) return hour;
        std::cout << "Invalid hour. Enter a whole number from 0 to 23.\\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
    }
}
void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0') << paise % 100
              << std::setfill(' ');
}
```

For money output, integer division gives whole rupees and the remainder gives paise. setw(2) and setfill('0') ensure two digits after the decimal point. fixed and setprecision(2) are appropriate when printing a double, but integer paise formatting is more exact and avoids floating-point output.

Sample journey report

Trip	Zone	Time	Peak?	Fare	Opening balance	Result / closing balance
1	Inner	08:30	Yes	Rs 12.00	Rs 100.00	Approved / Rs 88.00
2	Outer	10:00	No	Rs 30.00	Rs 88.00	Approved / Rs 58.00
3	Middle	18:15	Yes	Rs 24.00	Rs 58.00	Denied (would leave Rs 34.00)

Alternative concept and complexity analysis

With cin >>, formatted extraction reads a token and can leave the remaining line in the stream. Recovery requires clearing the fail state and discarding bad characters. With getline(), the whole line is read into a string and stoi() converts it; try/catch handles invalid text and extra validation is needed to ensure the entire line was consumed. Processing a line of length L is O(L) in either approach, and getline requires O(L) buffer space. getline gives precise line-level recovery, while cin >> is simpler for whitespace-separated numeric fields.

TASK 9 – Implement and Test the Fare Logic (4 Marks)

Testing must cover normal values, boundary values and invalid input. The expected values below use the original three-zone rules and assume the balance reserve is checked after fare deduction.

Test ID	Input	Expected fare / behaviour	Expected decision
T01	Inner, 07:59	Rs 10.00	Off-peak
T02	Inner, 08:00	Rs 12.00	Peak
T03	Middle, 09:59	Rs 24.00	Peak
T04	Middle, 10:00	Rs 20.00	Off-peak
T05	Outer, 17:00	Rs 36.00	Peak
T06	Outer, 19:59	Rs 36.00	Peak
T07	Outer, 20:00	Rs 30.00	Off-peak
T08	Unknown zone value	Invalid fare	Reject; balance unchanged
T09	Recharge = −100 paise	Invalid recharge	Reject; balance unchanged
T10	Hour input = abc	Input error	Clear and re-prompt
T11	Balance Rs 86; fare Rs 36	Closing balance Rs 50	Approve exactly at reserve
T12	Balance Rs 85.99; fare Rs 36	Would leave Rs 49.99	Deny; balance unchanged

Five-journey hand calculation

Journey	Zone / time	Calculation	Fare
1	Inner, 07:30	Rs 10.00	1,000 paise
2	Middle, 08:30	Rs 20 × 1.20	2,400 paise
3	Outer, 10:00	Rs 30.00	3,000 paise
4	Inner, 17:30	Rs 10 × 1.20	1,200 paise
5	Middle, 20:00	Rs 20.00	2,000 paise
Total		10 + 24 + 30 + 12 + 20	Rs 96.00

For an initial balance of Rs 200, the first four fares leave Rs 104. The fifth fare leaves Rs 84, which is above the Rs 50 reserve; all five journeys are approved. This sequence is a calculation example; the test runner should record actual outputs.

Table-driven test alternative and complexity

A table of test cases stores input values and expected fares, then a loop evaluates each case. For t tests, runtime is O(t), because each test performs a constant amount of work. The test table uses O(t) space.

This is easier to extend: a new boundary case is one additional row rather than another separately written test block. It also makes expected-versus-actual comparison systematic.

TASK 10 – Compare Expectations with Execution (4 Marks)

The following are mathematically derived totals for a deliberately simple upper-bound case in which every journey costs Rs 30.00. They are expected values, not claimed measurements from the student's machine.

Daily volume	Total rupees	Total paise	32-bit int paise	64-bit long long paise
20,000	Rs 600,000	60,000,000	Fits	Fits
500,000	Rs 15,000,000	1,500,000,000	Fits	Fits
3,000,000	Rs 90,000,000	9,000,000,000	Overflows	Fits

At 3,000,000 journeys, the total in paise is 9 billion, greater than the typical INT_MAX of 2,147,483,647. A signed int overflow is undefined behaviour in C++; do not rely on a wrapped result. long long has a much larger range and safely stores this total. double has a large range but can introduce small rounding differences.

Comparison of accumulation types

Type	Typical size	At 9,000,000,000 paise	Conclusion
int	4 bytes	Not representable; signed overflow is undefined behaviour.	Do not use for network totals.
long long	8 bytes	Exactly representable as an integer.	Recommended for paise totals.
double	8 bytes	Magnitude is representable; decimal fractions may be inexact.	Use only if approximation is acceptable, not for fare accounting.

Measurement method

```
#include <chrono>
#include <iostream>
int main() {
    constexpr long long N = 3000000;
    long long totalPaise = 0;
    auto start = std::chrono::steady_clock::now();
    for (long long i = 0; i < N; ++i) totalPaise += 3000;
    auto stop = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> elapsed = stop - start;
    std::cout << "Total paise: " << totalPaise << '\\n';
    std::cout << "Elapsed ms: " << elapsed.count() << '\\n';
}
```

Run with compiler optimisation settings recorded (for example g++ -std=c++17 -O2 -Wall -Wextra). Record your own elapsed time; timings vary by CPU, compiler, optimisation and background load. Do not invent a measured time. With -Wall -Wextra, warnings may identify signed/unsigned comparisons, narrowing or unused variables; exact warnings depend on the final code and compiler version. Correct the cause rather than suppressing warnings.

Each addition is O(1) for int, long long and double. Each accumulator uses O(1) space. The choice is governed by range and precision, not asymptotic complexity.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Recommended original design: use enum class Zone, constexpr named fare and policy constants, long long integer paise for all money, a Journey value type for trip details, a FareTable or fare-calculation service for policy, and a SmartCard class with private balance. Validate every input before fare calculation and use the exact half-open peak intervals.

Design decision	Recommendation	Reason
Money	long long paise	Exact whole-paise arithmetic and sufficient range.
Zone	enum class	Prevents accidental mixing of zone values and integers.
Fare constants	constexpr	One source of truth, compile-time checks.
Peak rule	One isPeak() function	Boundary behaviour is consistent at all gates.
Balance	Private SmartCard member	Protects the minimum reserve invariant.
Input	Validated input functions	Rejects invalid zone, hour and recharge values.
Testing	Table-driven tests	Repeatable regression and boundary testing.

A fare revision should be made in one central fare table or configuration source. The code should not contain separate hard-coded fare values in multiple functions. The limitations of the original design are that fixed constants require recompilation for every policy revision, and the three-zone switch must be updated when a zone is added. The original design remains computationally suitable for 3,000,000 journeys per day: each journey requires O(1) fare and balance operations, so processing n journeys is O(n), with O(1) additional working memory per journey.

Configuration-file alternative

If fares are read from a configuration file at gate startup, the program reads z zone fares in O(z) time and stores them in O(z) space. A validated file allows policy changes without recompiling the executable. The trade-off is additional startup I/O, parsing and error handling; the gate must refuse to start if a fare is missing, negative or malformed. For a stable small fare table, constexpr is simpler. For frequent policy changes, a signed and validated configuration file is more flexible.

TASK 12 – Re-evaluate and Adapt the Design: Day-4 Design Challenge (8 Marks)

Changed requirements: add Airport zone at Rs 60; apply a 10% weekend discount that does not combine with the peak surcharge; introduce a monthly pass allowing unlimited journeys in two selected zones.

A. Impact analysis (2 Marks)

Original element	Change required	Reason

Zone enum	Add Zone::Airport.	There are now four zones.
Base fare table	Extend from 3 to 4 values: 1000, 2000, 3000, 6000 paise.	Airport fare is Rs 60.
Switch statement	Add Airport case or replace switch with validated table lookup.	Old switch does not recognise fourth zone.
Peak expression	Keep the same time intervals; ensure weekend policy takes precedence.	Weekend discount must not combine with peak surcharge.
Journey	Add day type / weekend flag.	Fare depends on day as well as zone and time.
SmartCard	Retain private balance; add pass-aware approval logic or delegate to Pass.	Card safety invariant remains necessary.
New Pass class	Store two selected zones and validity period.	Unlimited travel in chosen zones is not a fare discount.

B. Evaluate the original design (2 Marks)

The original SmartCard class can be retained because private balance and controlled deduction remain useful. The original three-case switch can be modified by adding Airport, but the design becomes more fragile as zones grow. A fare table indexed by a validated enum is easier to extend. The original peak logic alone is insufficient because weekend discount must take precedence and must not be added to the peak surcharge.

Worked example	Correct revised result	Explanation
Airport, weekday, 18:00	Rs 72.00	Rs 60 + 20% peak surcharge.
Airport, weekend, 18:00	Rs 54.00	Weekend discount is 10%; peak surcharge is not applied.
Middle, weekend, 12:00	Rs 18.00	Rs 20 less 10% weekend discount.
Inner, weekday, 12:00	Rs 10.00	Normal off-peak weekday fare.
Monthly pass holder, valid pass, Middle zone	Rs 0.00 fare	Unlimited journey in one of the two selected zones, subject to pass validity.
Monthly pass holder, valid pass, Airport zone not selected	Normal fare	Pass covers only the two selected zones.

Assumption: the weekend discount applies to all fare-paying weekend journeys, and it replaces rather than combines with the weekday peak surcharge. The assignment does not define whether a monthly-pass journey should consume card balance or whether the Rs 50 reserve applies to pass-only journeys. This implementation treats a valid pass journey as zero fare and does not deduct balance; the Rs 50 reserve is checked only for fare-paying journeys. The metro authority should confirm this policy before deployment.

C. Adaptation / redesigned implementation (2 Marks)

The following C++17 design uses a four-entry fare table, a separate weekend rule and a Pass class. It validates the zone index before accessing the array. A pass covers exactly two distinct zones and is valid only when its validity flag is true.

```
#include <array>
#include <iostream>
#include <optional>

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Airport = 3 };
constexpr std::array<long long, 4> BASE_FARE{
    1000, 2000, 3000, 6000
};
constexpr long long MIN_BALANCE = 5000;

bool validZone(Zone z) {
    int i = static_cast<int>(z);
```

```
    return i >= 0 && i < static_cast<int>(BASE_FARE.size());
}

long long calculateFare(Zone z, int hour, bool weekend) {
    if (!validZone(z) || hour < 0 || hour > 23) return -1;
    long long fare = BASE_FARE[static_cast<int>(z)];
    if (weekend) {
        return fare * 90 / 100;       // weekend replaces peak rule
    }
    bool peak = (hour >= 8 && hour < 10) ||
                (hour >= 17 && hour < 20);
    if (peak) fare = fare * 120 / 100;
    return fare;
}

class Pass {
    Zone first;
    Zone second;
    bool valid;
public:
    Pass(Zone a, Zone b, bool isValid)
        : first(a), second(b), valid(isValid) {}
    bool covers(Zone z) const {
        return valid && (z == first || z == second);
    }
};

class SmartCard {
    long long balance;
public:
    explicit SmartCard(long long opening)
        : balance(opening >= 0 ? opening : 0) {}
    bool canPay(long long fare) const {
        return fare >= 0 && balance >= fare &&
               balance - fare >= MIN_BALANCE;
    }
    bool pay(long long fare) {
        if (!canPay(fare)) return false;
        balance -= fare;
        return true;
    }
    long long getBalance() const { return balance; }
};

bool processJourney(SmartCard& card, const Pass* pass,
                    Zone zone, int hour, bool weekend) {
    if (!validZone(zone)) return false;
    if (pass != nullptr && pass->covers(zone)) return true;
    long long fare = calculateFare(zone, hour, weekend);
    return card.pay(fare);
}
```

The revised design separates three concerns: zone-to-base-fare mapping, date/time fare policy, and pass entitlement. The SmartCard remains responsible for protecting the stored balance. A production implementation should additionally validate that the two pass zones are distinct and valid, store start/end dates, and handle recharge overflow.

D. Evidence-based defence (2 Marks)

Criterion	Switch-based original	Revised fare table + policy + Pass	Evaluation
Three-zone support	Simple and clear	Also supported	Both work.
Fourth zone	Requires another switch case	Add one table entry and enum value	Table is easier to extend.
Weekend rule	Needs extra conditional	Separate precedence rule	Revised design avoids stacking discounts.
Monthly pass	Not represented	Pass class checks two zones	Revised design models new requirement.
Balance safety	Private SmartCard balance	Private SmartCard balance retained	No loss of invariant.
Complexity	O(1) per fare; O(n) for n journeys	O(1) per fare/pass check; O(n) for n journeys	Same asymptotic performance.

Decision: MODIFY the original design rather than discard it. Retain SmartCard encapsulation and integer-paise accounting; replace the small switch-only fare mapping with a four-entry validated fare table; isolate weekend/peak precedence in calculateFare(); and add Pass for entitlement checks. This approach meets the changed requirements with O(1) fare calculation and O(1) pass checking per journey. The design is easier to extend and keeps the balance invariant protected.
Evidence to attach after execution: output screenshots for weekday peak Airport (Rs 72), weekend Airport (Rs 54), weekend Middle (Rs 18), valid pass-covered zone (Rs 0), uncovered zone (normal fare), invalid zone and balance exactly Rs 50. These are expected results from the code and assumptions above; insert screenshots from your own run.

APPENDIX A – Complete Integrated C++17 Fare Program

This integrated example combines the original three-zone fare rules, safe validation, exact paise arithmetic and SmartCard encapsulation. It is intended as a clear baseline for Tasks 6–9. The separate Task 12 implementation extends it for the changed requirements.

```
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>

enum class Zone { Inner, Middle, Outer };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

long long calculateFare(Zone zone, int hour) {
    if (hour < 0 || hour > 23) return -1;
    long long base = 0;
    switch (zone) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (isPeak(hour))
        base = base * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    return base;
}

class SmartCard {
private:
    long long balancePaise;
public:
    explicit SmartCard(long long openingBalance)
        : balancePaise(openingBalance >= 0 ? openingBalance : 0) {}

    long long getBalance() const { return balancePaise; }

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }
```

```
    bool recharge(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        return true;
    }
};

void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0') << paise % 100
              << std::setfill(' ');
}

int main() {
    SmartCard card(10000);
    struct Trip { Zone zone; int hour; };
    const Trip trips[] = {
        {Zone::Inner, 7},
        {Zone::Outer, 18},
        {Zone::Middle, 12}
    };

    int tripNo = 1;
    for (const Trip& trip : trips) {
        long long fare = calculateFare(trip.zone, trip.hour);
        std::cout << "Trip " << tripNo++ << ": ";
        if (fare < 0) {
            std::cout << "Invalid journey; ";
        } else if (card.deduct(fare)) {
            std::cout << "Approved; fare ";
            printMoney(fare);
            std::cout << "; ";
        } else {
            std::cout << "Denied (low balance); ";
        }
        std::cout << "balance ";
        printMoney(card.getBalance());
        std::cout << '\\n';
    }
    return 0;
}
```

Compile command (run from the folder containing the file):

```
g++ -std=c++17 -Wall -Wextra -Wconversion -pedantic metro_fare.cpp -o metro_fare
```

Run command:

```
./metro_fare
```

Windows PowerShell run command:

```
.\\metro_fare.exe
```

Appendix B – GitHub submission and evidence checklist

- Create repository named DSA01-<RegisterNumber>-Assignment1.
- Create Task01 through Task12 folders (or Q1 through Q12).
- Save each task's .cpp source file in its matching folder.
- Save relevant input files and genuine terminal/output screenshots.

- Include README.md with student name, register number, g++ version, exact compile command, run steps and short result for each task.
- Commit progressively: at least one meaningful commit per task; do not upload everything in one final commit.
- Add the course faculty as collaborator if the repository is private, or make it public if permitted.
- Paste the working GitHub URL on the cover page and in the assignment submission field.
- Run all code with -Wall -Wextra; fix warnings and record any remaining warning with explanation.
- Replace illustrative timing statements with actual measurements from your own system.
- Be prepared to explain every class, conversion, boundary condition, test and complexity claim in the viva.